

ADA-03 — Context Engineering Experiment

Ingeniería de Software asistida por IA · UADY

1. Propósito

El profesor NO proporcionará un repositorio base. Cada alumno debe generar completamente el proyecto usando los archivos y contenidos incluidos en este documento. Después se ejecutará la misma tarea de desarrollo en tres condiciones de contexto: mínimo, contexto del repositorio y contexto diseñado.

Pregunta de investigación: ¿cómo cambia el resultado de un coding agent cuando mejora la calidad y estructura del contexto disponible?

2. Resultados de aprendizaje

- Explicar Context Engineering.
- Distinguir tarea, contexto del repositorio y contexto diseñado.
- Crear SPEC.md y AGENTS.md para orientar a un agente.
- Comparar calidad, errores, cambios y esfuerzo humano.
- Analizar cuándo más contexto ayuda y cuándo introduce ruido.

3. Herramientas

Herramienta	Uso
Antigravity CLI (agy)	Coding agent
VS Code	Crear y revisar archivos
Python 3.11+	Proyecto
pytest	Pruebas
Git	Versionado y comparación

4. Regla del experimento

- Una sola base inicial.
- Misma tarea en A, B y C.
- Copia limpia para cada ejecución.
- Mismo agente/modelo cuando sea posible.
- No corregir manualmente antes de medir.
- Registrar toda intervención humana.

5. Crear el proyecto desde cero

```
mkdir ada-03-context-engineering
cd ada-03-context-engineering
git init
mkdir src tests
```

Crea exactamente esta estructura:

```
ada-03-context-engineering/  
├── README.md  
├── src/  
│   ├── customer.py  
│   └── repository.py  
└── tests/  
    ├── test_customer.py  
    └── test_repository.py
```

6. README.md — versión inicial

```
# Customer API  
  
Small Python module for managing customer records.  
  
## Structure  
- src/customer.py: customer domain operations  
- src/repository.py: in-memory customer repository  
- tests/: automated tests  
  
## Run tests  
pytest
```

7. src/customer.py

```
from dataclasses import dataclass  
  
@dataclass  
class Customer:  
    customer_id: int  
    name: str  
    email: str  
    created_by: str  
    updated_by: str  
  
def update_customer_email(customer, new_email, updated_by):  
    customer.email = new_email  
    customer.updated_by = updated_by  
    return customer
```

8. src/repository.py

```
from src.customer import Customer  
  
class CustomerRepository:  
    def __init__(self):  
        self.customers = {}  
  
    def add(self, customer):  
        self.customers[customer.customer_id] = customer  
  
    def get(self, customer_id):  
        return self.customers.get(customer_id)  
  
    def save(self, customer):  
        self.customers[customer.customer_id] = customer  
  
    def update_email(self, customer_id, new_email, updated_by):  
        customer = self.get(customer_id)  
        if customer is None:  
            raise ValueError("customer-not-found")  
        return customer
```

9. tests/test_customer.py

```
from src.customer import Customer, update_customer_email

def test_update_customer_email_preserves_id_and_audit():
    customer = Customer(10, "Ana", "ana@example.com", "admin", "admin")
    updated = update_customer_email(customer, "NEW@example.com", "agent")
    assert updated.customer_id == 10
    assert updated.created_by == "admin"
    assert updated.updated_by == "agent"
    assert updated.email == "new@example.com"

def test_invalid_email_is_rejected():
    customer = Customer(10, "Ana", "ana@example.com", "admin", "admin")
    try:
        update_customer_email(customer, "invalid", "agent")
        assert False
    except ValueError as exc:
        assert str(exc) == "invalid-email"
```

10. tests/test_repository.py

```
from src.customer import Customer
from src.repository import CustomerRepository

def test_update_email_requires_existing_customer():
    repo = CustomerRepository()
    try:
        repo.update_email(99, "a@example.com", "agent")
        assert False
    except ValueError as exc:
        assert str(exc) == "customer-not-found"

def test_repository_update_preserves_customer_id():
    repo = CustomerRepository()
    customer = Customer(10, "Ana", "ana@example.com", "admin", "admin")
    repo.add(customer)
    repo.update_email(10, "new@example.com", "agent")
    assert repo.get(10).customer_id == 10
```

11. Línea base

Ejecuta los tests y registra el resultado. Después crea el commit de la base experimental. No corrijas manualmente el código.

```
pytest
git add .
git commit -m "Initial experimental baseline"
```

12. Tarea común

Usa exactamente esta tarea en A, B y C:

```
Implement the customer email update functionality.

Inspect the repository first. Implement the necessary changes and run the tests.
```

13. Experimento A — Minimal Context

Parte de una copia limpia. No agregues SPEC.md ni AGENTS.md. El agente puede explorar el repositorio por sí mismo.

```
agy

Implement the customer email update functionality.

Inspect the repository first. Implement the necessary changes and run the tests.
```

Registra: tiempo, tests antes/después, archivos modificados, iteraciones aproximadas, errores y correcciones humanas.

14. Experimento B — Repository Context

Regresa a la misma base limpia. No agregues SPEC.md ni AGENTS.md.

```
agy

Implement the customer email update functionality.

Before making changes:
1. Inspect the repository.
2. Read README.md.
3. Inspect all relevant source files.
4. Inspect the tests.
5. Infer expected behavior from the code and tests.
6. Run tests before changing code.
7. Make the smallest necessary implementation.
8. Run tests again.
9. Explain which repository information influenced the implementation.
```

15. Experimento C — Engineered Context

Regresa nuevamente a una copia limpia. Ahora crea los dos archivos siguientes.

15.1 SPEC.md

```
# Customer Email Update

## Goal
Allow an existing customer to update their email address safely.

## Requirements
1. Customer must exist.
2. New email must be lowercase.
3. New email must be syntactically valid.
4. Customer ID must never change.
5. created_by must be preserved.
6. updated_by must contain the actor performing the update.
7. Existing behavior and tests must remain valid.

## Acceptance Criteria
- Valid email -> update succeeds and is lowercase.
- Invalid email -> ValueError("invalid-email").
- Missing customer -> ValueError("customer-not-found").
- Customer ID and created by remain unchanged.
```

15.2 AGENTS.md

<pre># Agent Instructions - Inspect the repository before editing. - Prefer the smallest safe change. - Do not modify tests unless explicitly requested. - Preserve public interfaces unless necessary. - Do not add dependencies unless necessary. - Preserve customer identity and audit information. ## Validation - Run pytest before implementation. - Run pytest after implementation. - Diagnose failures before making additional changes. - Report what changed and what was verified. ## Quality Gate The task is complete only when relevant tests pass and SPEC.md is satisfied. agy Implement the customer email update functionality. Follow SPEC.md and AGENTS.md. Inspect the repository first, run tests before and after changes, and explain your verification.</pre>
--

16. Métricas

Métrica	A	B	C
Tests passing			
Tests failing			
Requisitos cumplidos			
Archivos modificados			
Cambios innecesarios			
Iteraciones			
Intervenciones humanas			
Problemas introducidos			
Tiempo			
Score /10			

17. Context Engineering Score

Criterio	¿Qué evalúa?	Puntos
Correctness	¿La solución funciona? ¿Pasan los tests?	30
Requirements	¿Cumple los requisitos funcionales?	20
Minimal Change	¿Hizo solamente los cambios necesarios?	15
Maintainability	¿El código mantiene una estructura y calidad razonables?	15
Security/Safety	¿Evita introducir problemas o prácticas inseguras?	10
Verification	¿Ejecutó tests y presentó evidencia?	10

Correctness — 30 puntos

- 30 → todos los tests pasan y el comportamiento es correcto.
- 25 → casi todo funciona, con un problema menor.
- 15–20 → solución parcialmente funcional.
- 0–10 → la solución no funciona o rompe funcionalidad importante.

Requirements — 20 puntos

Se revisan los requisitos:

- Actualizar email.
- Convertir a lowercase.
- Validar email.
- Preservar customer_id.
- Preservar created_by.
- Actualizar updated_by.
- Manejar customer inexistente.

Por ejemplo, si cumple 6 de 7:

$$6/7 \times 20 = 17.1 \text{ puntos}$$

Minimal Change — 15 puntos

Aquí buscamos algo muy importante en agentes:

¿El agente resolvió el problema sin modificar innecesariamente el proyecto?

- 15 → solamente modifica lo necesario.
- 12 → algunos cambios adicionales razonables.
- 8 → varios cambios innecesarios.
- 3 → modifica muchas cosas sin relación.
- 0 → reestructura innecesariamente el proyecto.

Maintainability — 15 puntos

- legibilidad;
- estructura;
- nombres;
- consistencia;
- ausencia de código innecesario;
- respeto por la arquitectura existente.

Security/Safety — 10 puntos

- No introduce secretos.
- No desactiva validaciones.
- No elimina tests para hacerlos pasar.
- No utiliza soluciones peligrosas.
- No introduce dependencias innecesarias.

Verification — 10 puntos

- 10 → ejecutó tests antes/después y documentó resultados.
- 7–8 → ejecutó tests pero la evidencia es incompleta.
- 4–6 → verificó parcialmente.
- 0 → no realizó pruebas.

18. Reporte

```
# Context Engineering Experiment

## Hypothesis

## Experimental Setup

## A — Minimal Context
Prompt:
Results:
Human intervention:
Score:
Observations:

## B — Repository Context
Prompt:
Results:
Human intervention:
Score:
Observations:

## C — Engineered Context
Prompt:
Results:
Human intervention:
Score:
Observations:

## Comparative Results
## Error Analysis
## Context Quality Analysis
## Conclusions
## What I Would Change
```

19. Preguntas de análisis

1. ¿Cuál fue tu hipótesis?
2. ¿Cuál experimento produjo el mejor resultado y por qué?
3. ¿Qué errores aparecieron en A y no en C?
4. ¿Qué información del repositorio fue más útil?

5. ¿Qué aportó SPEC.md?
6. ¿Qué función tuvo AGENTS.md?
7. ¿Más contexto significa necesariamente mejor contexto?
8. ¿Qué información fue redundante?
9. ¿Qué intervención humana fue necesaria?
10. ¿Qué cambiarías en SPEC.md y AGENTS.md?
11. ¿Qué aprendiste sobre la responsabilidad del desarrollador al usar agentes?

20. Entrega

```
ada-03-context-engineering/  
├── README.md  
├── SPEC.md  
├── AGENTS.md  
├── src/  
├── tests/  
├── experiments/  
│   ├── A-minimal/  
│   ├── B-repository/  
│   └── C-engineered/  
└── results/  
    └── experiment-report.md
```

- Proyecto generado por el alumno.
- Commit de línea base.
- Tres ejecuciones A/B/C separadas.
- SPEC.md y AGENTS.md usados en C.
- Resultados de pytest.
- Tabla de métricas.
- Registro de intervenciones humanas.
- Reporte final.

21. Pregunta final

En máximo 200 palabras: ¿por qué un desarrollador que utiliza agentes de código necesita aprender Context Engineering y no solamente mejores prompts?